Learning-Based UAV Path Planning for Data Collection With Integrated Collision Avoidance

Xueyuan Wang, M. Cenk Gursoy, *Senior Member, IEEE*, Tugba Erpek, and Yalin E. Sagduyu, *Senior Member, IEEE*

***Abstract*—Unmanned aerial vehicles (UAVs) are expected to be an integral part of wireless networks, and determining collision-free trajectory in multi-UAV noncooperative scenarios while collecting data from distributed Internet of Things (IoT) nodes is a challenging task. In this article, we consider a path-planning optimization problem to maximize the collected data from multiple IoT nodes under realistic constraints. The considered multi-UAV noncooperative scenarios involve a random number of other UAVs in addition to the typical UAV, and UAVs do not communicate or share information among each other. We translate the problem into a Markov decision process (MDP) with parameterized states, permissible actions, and detailed reward functions. Dueling double deep *Q*-network (D3QN) is proposed to learn the decision-making policy for the typical UAV, without any prior knowledge of the environment (e.g., channel propagation model and locations of the obstacles) and other UAVs (e.g., their missions, movements, and policies). The proposed algorithm can adapt to various missions in various scenarios, e.g., different numbers and positions of IoT nodes, different amount of data to be collected, and different numbers and positions of other UAVs. Numerical results demonstrate that real-time navigation can be efficiently performed with high success rate, high data collection rate, and low collision rate.**

***Index Terms*—Collision avoidance, data collection, deep reinforcement learning (RL), multiunmanned aerial vehicle (UAV) scenarios, path planning.**

I. Introduction

UNMANNED aerial vehicles (UAVs), also commonly known as drones, are aircrafts piloted by remote control or embedded computer programs without human onboard [1]. UAVs have been one of the main targets of industrial and academic research in recent years. Due to their mobility, autonomy and flexibility, UAVs can be used in a variety of real-world scenarios, such as delivery of medical supplies, disaster relief, environment monitoring, aerial surveillance and inspection, traffic control, and emergency search and rescue [2], [3]. In order to take advantage of flexible deployment opportunities and high possibility of Line-of-Sight (LoS) connections with ground user equipments (UEs), UAVs can be deployed as base stations (BSs) to support wireless connectivity and improve the performance of cellular networks [4], leading to a UAV-assisted cellular network architecture. UAVs have shown particular promise in collecting data from distributed Internet of Thing (IoT) sensor nodes, as IoT operators can deploy UAV data harvesters in the absence of other expensive cellular infrastructure nearby [5]. For example, UAVs can move toward potential ground nodes and establish reliable connections with low transmit power, and hence they can provide an energy-efficient solution for data collection from ground UEs that are spread over a geographical area with limited terrestrial infrastructure [6]. However, optimizing the flight path of UAVs is challenging as it requires the consideration of many physical constraints and parameters. In particular, the trajectory of a UAV is significantly affected by different factors such as flight time, kinematic constraints, ground UEs demands, and collision avoidance [3]. Effective UAV trajectory planning allows the UAVs to adapt their movement based on the communication requirements of both the UAVs and the ground UEs, thus improving the overall network performance [7] and, therefore, UAV path plans and control policies need to be carefully designed such that the application requirements are satisfied [8]–[10].

A. Related Prior Work

UAV trajectory design for data collection in IoT networks has been intensively studied in the literature. For example, to minimize the weighted sum of the propulsion energy consumption and operation costs of all UAVs and the energy consumption of all sensor nodes, the nodes' wake-up time allocation and the transmit power and the UAV trajectories were jointly optimized in [11], and collision-avoidance constraint was also considered in this article. Wang *et al.* [12] aimed to minimize the energy consumption of IoT devices by jointly optimizing the UAV trajectory and device transmission scheduling over time. In [13], the UAV trajectory, altitude, velocity and data links with the ground UEs were optimized to

Manuscript received 17 December 2021; accepted 7 February 2022. Date of publication 23 February 2022; date of current version 24 August 2022. *(Corresponding author: M. Cenk Gursoy.)*

Xueyuan Wang was with the Department of Electrical Engineering and Computer Science, Syracuse University, Syracuse, NY 13244 USA. She is now with the School of Computer Science and Artificial Intelligence, Changzhou University, Changzhou 213164, China (e-mail: xwang173@syr.edu).

M. Cenk Gursoy is with the Department of Electrical Engineering and Computer Science, Syracuse University, Syracuse, NY 13244 USA (e-mail: mcgursoy@syr.edu).

Tugba Erpek and Yalin E. Sagduyu are with the National Security Institute, Virginia Tech, Arlington, VA 22201 USA (e-mail: terpek@vt.edu; yalinsagduyu@gmail.com).

Digital Object Identifier 10.1109/JIOT.2022.3153585

2327-4662 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

minimize the total mission time for UAV-aided data collection. Samir *et al.* [14] aimed to optimize the UAV trajectory and the radio resource allocation to maximize the number of served IoT devices, where each device has a data upload deadline. The minimum throughput over all ground UEs in downlink communication was maximized in [15], by optimizing the multiuser communication scheduling and association jointly with the UAV's trajectory and power control. Moreover, UAV path planning for data collection has also been investigated in [16]–[19]. Nonlearning-based algorithms and optimization techniques, e.g., block coordinate descent, successive convex approximation, and alternating optimization, were proposed and utilized to solve the problems in these studies.

Reinforcement learning (RL) is the study of how an agent can interact with the environment to learn a policy that maximizes the expected cumulative reward for a task [20]. In the areas of communications and networking, RL has been recently used as an emerging tool to effectively address various problems and challenges. Specifically, RL-based algorithms have been proposed as solutions to UAV path planning for data collection tasks. For example, [21]–[26] addressed the trajectory optimization in single-UAV data collection scenarios. Yi *et al.* [21] used double deep Q-network (DDQN) algorithms to find the optimal flight trajectory and transmission scheduling to minimize the weighted sum of the age of the information. Zhang *et al.* [22] used deep Q-network (DQN) algorithms to decide the UAV trajectory to collect the required data, and determined the charging car trajectory to reach its destination to charge the UAV. DQN was also used in [23] to decide the transmission schedule to minimize the data loss, given the waypoint of the UAV's trajectory. Bouhamed *et al.* [24] first used a deep deterministic policy gradient (DDPG) algorithm to find the trajectory with no collision with obstacles, and then employed Q-learning (QL) to find the scheduling strategy to minimize the data collection time. Fu *et al.* [25] provided a QL framework as an energy-efficient solution for the UAV trajectory optimization. Moreover, QL was also used in [26] to find the UAV trajectory to maximize the sum rate of transmissions.

Different RL-based path-planning approaches for data collection in multi-UAV networks have been developed in the literature as well, such as in [5] and [27]–[30]. Particularly, Bayerlein *et al.* [5] utilized the DDQN algorithm to solve the UAV path-planning problem to maximize the collected data from IoT nodes, subject to flying time and collision-avoidance constraints. However, the learning was centralized, and the UAVs needed to share their information and a part of their reward among each other. Hsu and Gau [27] considered a scenario where the UAVs took charge of delivering objects in the forward path, and collected data from IoT devices in the backward path. QL was used to solve the forward collision-avoidance problem, and auxiliary no-return traveling salesman algorithm was used to find the shortest backward path. However, the collision-avoidance and communication constraints were not taken into account together. Moreover, Liu *et al.* [28] considered the problem of joint trajectory design and power control for multi-UAVs for maximizing the instantaneous sum transmit rate of mobile UEs. To solve the problem, a framework that involves a multiagent QL-based placement algorithm for initial deployment of UAVs, an echo-state-network-based algorithm for predicting the mobility of UEs, and a multiagent QL-based trajectory-acquisition and power control algorithm for UAVs, was proposed. However, collision-avoidance constraint was not taken into account in this article. Vehicles and UAVs were considered to collect data from multiple IoT nodes cooperatively in [29] without accounting for collision-avoidance constraints. Genetic algorithm was utilized to select vehicular collectors, and Asynchronous Advantage Actor–Critic (A3C) algorithm was employed to plan collection routes of UAVs subject to energy constraints. In [30], a joint trajectory design and power allocation problem was considered, and a multiagent DDPG method was proposed to obtain the optimal policy under UEs' quality of service requirements. In these studies, all agents were assumed to use the same policy or operate cooperatively. The key challenge for these models is that they cannot generalize well to crowded scenarios (in which different policies are typically adopted) or decentralized settings with noncooperating agents.

B. Contributions

In this article, we propose and implement a dueling DDQN (D3QN)-based path-planning algorithm for data collection in multi-UAV noncooperative decentralized scenarios. The objective for a typical UAV is to plan a collision-free trajectory to destination and collect data from multiple distributed IoT nodes, based on the information sensed from the environment and received from IoT nodes. The main contributions are summarized as follows.

1) We study the UAV trajectory optimization to maximize the collected data from distributed IoT nodes in multi-UAV noncooperative scenarios under realistic constraints, e.g., collision-avoidance, mission completion deadline, and kinematic constraints. The considered multi-UAV noncooperative scenarios involve random number of other UAVs in addition to the typical UAV, and UAVs do not communicate and share information among each other. The typical UAV can only observe nearby UAVs in its sensing region.
2) Due to the uncertainty in the environment, other UAVs' existence and unobservable intents, the considered problem is translated to a Markov decision process (MDP) with parameterized states, permissible actions, and detailed reward functions. D3QN framework is proposed for learning the policy without any prior knowledge of the environment (e.g., channel propagation model and locations of the obstacles) and other UAVs (e.g., their missions, intents, and policies). The proposed D3QN RL algorithm is tailored to the UAV path-planning problem by meticulously designing the state space, action space, and reward mechanisms in a novel way to accurately reflect the objectives and constraints.
3) The proposed algorithm has high adaptation capability. More specifically, without further training, the offline learned policy can be used for real-time navigation for

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

various missions with different number and locations of IoT nodes, different amount of data to be collected, in various scenarios, including different number and locations of other UAVs.

4) The proposed algorithm operates in a decentralized fashion without requiring cooperation. Particularly, the UAVs do not communicate or share any information among each other, and other UAVs may have different missions and use different policies (which is unknown to the typical UAV). Regardless of the mission, intents, and decision making policies, as long as nearby UAVs are sensed, the typical UAV can adjust its movements accordingly using a learned policy.
5) We extensively evaluate the proposed D3QN path-planning algorithm. We show that real-time navigation can be efficiently performed with high success rate (SR) and high data collection rate (DR) for various missions in various scenarios (including crowded ones). We demonstrate that the proposed algorithm can achieve much lower collision rate (CR) in testing compared with the algorithm not considering collision avoidance. For example, in scenarios with 20 other UAVs, if collision avoidance is not considered, the CR rises up to 29.1%, while it is only 0.6% with the proposed algorithm. In addition, the proposed algorithm has high tolerance to noisy observations (as demonstrated in Section IV-E). Furthermore, we show that D3QN has better performance than Dueling DQN, DDQN, DQN, and nodes-as-waypoints algorithm when solving the considered problem.

The remainder of this article is organized as follows: Section II provides the details of the considered system model, and formulates the path-planning optimization problem for data collection in multi-UAV scenarios. Section III describes the RL framework and presents the details of the proposed D3QN path planning algorithm. Section IV focuses on numerical and simulation results to evaluate the performance of the proposed algorithm. Finally, concluding remarks are provided in Section V.

II. System Model and Problem Formulation

In this section, we first introduce the system model in detail, and then we formulate the path-planning optimization problem for data collection.

A. System Model

We assume that the area of interest is a cubic volume, which can be specified by $\mathbb{C} : \mathbb{X} \times \mathbb{Y} \times \mathbb{Z}$ and $\mathbb{X} \triangleq [x_{\min}, x_{\max}]$, $\mathbb{Y} \triangleq [y_{\min}, y_{\max}]$, and $\mathbb{Z} \triangleq [z_{\min}, z_{\max}]$. There are multiple no-fly zones (obstacles) in the area through which UAVs cannot fly. The no-fly zones are denoted as $\mathbb{N} : \mathbb{X}^N \times \mathbb{Y}^N \times \mathbb{Z}$.

1) UAV: In the considered multi-UAV scenario, we choose one UAV as the typical one, whose mission is to collect data from multiple ground IoT nodes. The UAV is modeled as disc-shaped with radius r. Let $\mathbf{p} = [p_x, p_y, H_V]$ denote the 3-D position of the UAV, where H_V is the altitude of the UAV which is assumed to be fixed. It is further assumed that the typical UAV has specific areas for departure and landing, which can be denoted by $\mathbb{S}$ and $\mathbb{D}$, respectively. More specifically, $\mathbb{S} : \mathbb{X}^S \times \mathbb{Y}^S \times \mathbb{Z}$, where $\mathbb{X}^S \triangleq [x^S_{\min}, x^S_{\max}]$, $\mathbb{Y}^S \triangleq [y^S_{\min}, y^S_{\max}]$, and $\mathbb{D} : \mathbb{X}^D \times \mathbb{Y}^D \times \mathbb{Z}$, where $\mathbb{X}^D \triangleq [x^D_{\min}, x^D_{\max}]$, $\mathbb{Y}^D \triangleq [y^D_{\min}, y^D_{\max}]$. $\mathbf{p}^S = [p_{sx}, p_{sy}, H_V]$ and $\mathbf{p}^D = [p_{gx}, p_{gy}, H_V]$ are used to denote the coordinates of the starting point and the destination for the typical UAV, respectively. The typical UAV's information forms a vector that consists of the UAV's position, current velocity $\mathbf{v} = [v_x, v_y]$, radius r, destination $\mathbf{p}^D$, maximum speed $v_{\max}$, and orientation ϕ, i.e., $\mathbf{s} = [\mathbf{p}, \mathbf{v}, r, \mathbf{p}^D, v_{\max}, \phi] \in \mathbb{R}^{11}$.

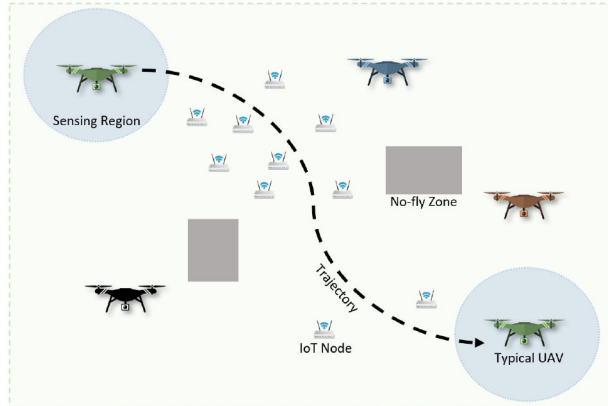


Fig. 1. Illustration of data collection in a multi-UAV scenario.

In this multi-UAV scenario, there are also J other UAVs traveling within region C. None of the UAVs communicate with each other. Therefore, the missions, destinations, movements, and decision-making policies of other UAVs are unknown. It is assumed that the typical UAV is equipped with a sensor, with which it is able to sense the existence of other UAVs when they are closer than a certain distance. The circular sensing region is denoted by $\mathbb{O}$. Specifically, if the jth UAV is in $\mathbb{O}$, some information of this UAV can be known by the typical UAV. The observable information includes the jth UAV's position $\mathbf{p}_j = [p_{x_j}, p_{y_j}, H_V]$, current velocity $\mathbf{v}_j = [v_{x_j}, v_{y_j}]$, and radius r_j, i.e., $\mathbf{s}^o_j = [\mathbf{p}_j, \mathbf{v}_j, r_j] \in \mathbb{R}^6$. The total number of other UAVs in $\mathbb{O}$ is denoted by J^o. It is worth noting that J^o varies over time.

2) IoT Nodes: In this UAV-assisted network, there are N static IoT nodes that need to upload data to the typical UAV via uplink transmission. The nth node has transmit power P_n, and is located at ground position $\mathbf{p}_n = [p_{x_n}, p_{y_n}]$. Each node has a finite amount of data D^L_{n0} that needs to be collected over the entire mission duration of the typical UAV. It is assumed that $\mathbf{p}_n$ and D^L_{n0} are known by the typical UAV in advance before a mission. The IoT nodes have two modes: active mode, if the node still has data to transmit; and silent mode, if data upload is completed.

An illustration of the system model is provided in Fig. 1. Notations of key parameters are summarized in Table I.

B. Channel Model

1) Path Loss: Due to high UAV altitude, air-to-ground channels usually constitute strong LOS links, and hence LOS

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TABLE OF NOTATIONS

Notations	Description
$\mathbb{C}$, $\mathbb{N}$	Area of interest; no-fly zones/obstacles
$\mathbb{O}$	The typical UAV's sensing region
N, J	The number of IoT nodes; the number other UAVs
J^o	Number of UAVs in the typical UAV's sensing region
$\mathbf{p}, \mathbf{v}, \phi$	UAV's position, velocity, orientation
r, H_V	UAV's radius, altitude
P_n, G_n	Transmit power and antenna gain of the nth IoT node
G_V, L	The UAV's antenna gain, path loss
P_n^r	Received signal power from the nth IoT node
q_n	Connection indicator with the nth IoT node
S_n, R_n	SNR and rate when connected with the nth node
$\mathcal{T}_s, \mathcal{T}_t$	SNR threshold, mission completion time threshold
$v_{\max}$, $\mathcal{T}_r$	Maximum speed, maximum rotation angle in unit time
$\mathcal{N}_s$	The noise power
$\mathcal{S}, \mathcal{A}, \mathcal{R}$	State space, action space, reward
Δt	One time step duration
T	Number of time steps

links are dominant [1], [2]. In addition, if UAV's altitude H_V is greater than a threshold, 3GPP specifications suggest a LOS link with probability 1. For example, as stated in the 3GPP specifications in [31], the altitude threshold is suggested to be 40 m for RMa (Rural Macro) deployment, and 100 m for UMa (Urban Macro) deployment. Therefore, we assume that all links between the UAV and IoT nodes are LOS. Then, the path loss can be expressed as

$$L(d) = \left(d^2 + H_V^2\right)^{\alpha/2} \tag{1}$$

where d is the horizontal distance between the ground projection of the UAV and a node, and α is the path-loss exponent.

2) Antenna Configuration: The IoT nodes are assumed to have the omnidirectional antenna gain of $G_n = 0$ dB. The UAVs are assumed to be equipped with a receiver with a horizontally oriented antenna, and a simple analytical approximation for antenna gain provided by UAV can be expressed as [32]

$$G_V(d) = \sin(\theta) = \frac{H_V}{\sqrt{d^2 + (H_V)^2}} \tag{2}$$

where θ is elevation angle between the UAV and a node. An illustration of the UAV's antenna pattern is given in Fig. 2. While we specify a certain antenna pattern here, the remainder of the analysis is applicable to any antenna pattern.

C. Signal-to-Noise Ratio and Rate

The received signal from the nth node at the typical UAV can be expressed as $P_n^r = P_n G_V(d_n) L^{-1}(d_n)$. With this, the experienced signal-to-noise ratio (SNR) at the UAV if it is communicating with the nth IoT node can be formulated as

$$S_n \triangleq \frac{P_n}{\mathcal{N}_s} G_V(d_n) L^{-1}(d_n) = \frac{P_n}{\mathcal{N}_s} H_V \left(d^2 + H_V^2\right)^{-\frac{1+\alpha}{2}} \tag{3}$$

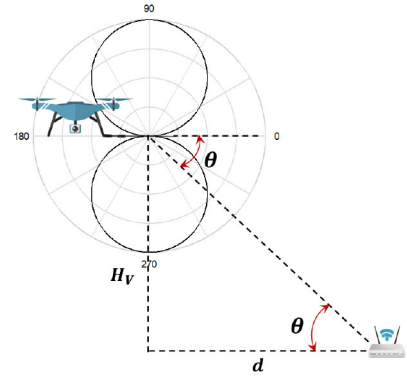


Fig. 2. Illustration of the UAV's antenna pattern.

where $\mathcal{N}_s$ is the noise power. The maximum achievable information rate if the UAV is connected with the nth node is

$$R_n^{\max} = \log_2(1 + S_n). \tag{4}$$

To support data flows, UAV has to maintain a reliable communication link to the IoT nodes. To achieve this, it is assumed that the experienced SNR at the UAV when connecting with a node should be larger than a certain threshold $\mathcal{T}_s$. Then, the UAV can communicate with the node successfully. Otherwise, the UAV is not able to collect data from the node. Therefore, the effective information rate according to the SNR threshold $\mathcal{T}_s$ can be given as

$$R_n = \begin{cases} R_n^{\max}, & \text{if } S_n \geq \mathcal{T}_s \\ 0, & \text{otherwise.} \end{cases} \tag{5}$$

D. Scheduling

Since the typical UAV needs to communicate with multiple nodes, we adopt the standard time-division multiple access (TDMA) model. Then, the UAVs can communicate with at most one node at each time. Using $q_n \in \{0, 1\}$ to indicate the connection with the nth node, we have

$$\sum_{n=1}^{N} q_n \leq 1. \tag{6}$$

The scheduling is according to the largest received signal power strategy, meaning that the UAV is connected with the active node providing the largest P_n^r. We can mathematically express the scheduling strategy as

$$q_n = \begin{cases} 1, & \text{if } n = \operatorname{argmax}_{n' \in \{\text{active nodes}\}} P_{n'}^r \\ 0, & \text{otherwise.} \end{cases} \tag{7}$$

E. Problem Formulation

We can partition each mission duration in the discrete-time domain to a number of time steps $t \in [0, T]$, with each time step describing a period of Δt. Now, the integer-valued t is used to denote each time step. We next consider the following realistic and practical constraints in the design of UAV trajectories:

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

1) Collision-Avoidance Constraints: In scenarios involving multiple UAVs, a fundamental challenge is to safely control the interactions with other dynamic agents in the environment. Therefore, for collision-avoidance purposes, the minimum distance between the typical UAV and any other UAVs should not be smaller than the sum of their radii at all times. In addition, it is important for UAVs to navigate while staying free of collisions with fixed obstacles and avoiding no-fly zones. In this setting, we can write the collision-avoidance constraints as

$$|\mathbf{p}_t - \mathbf{p}_{jt}||_2 > r + r_j \quad \forall j \quad \forall t \tag{8}$$

$$\mathbf{p}_t \notin \mathbb{N} \quad \forall t \tag{9}$$

where $\mathbf{p}_t$ is the position of the typical UAV at time step t, $\mathbf{p}_{jt}$ is the position of the jth UAV at time t, and r and r_j are the radii of typical UAV and the jth other UAV, respectively. Equation (9) is to avoid collision with the obstacles and avoid no-fly zones.

We note that unmanned vehicles/systems are typically equipped with sensors (including, e.g., laser ranging, radar, electro-optical, infrared, or thermal imaging sensors, and motion detectors) for safe and robust operation. UAVs, equipped with one or two types of such sensors, will have the capability to detect the observable state of the nearby UAVs at low cost.

2) Mission Completion Deadline Constraint: A UAV with a mission has to complete the required tasks in a certain time period. Moreover, finite amount of energy available at the battery-operated UAV also restricts the UAV's flight time. Hence, we assume that the UAV has a mission completion deadline constraint that can be described as

$$T \cdot \Delta t \leq \mathcal{T}_t \tag{10}$$

where T is the total steps in discrete-time domain, and $\mathcal{T}_t$ is the maximum allowed mission completion time.

3) Kinematic Constraints: In practice, the kinematic constraints should be considered in operating UAVs. Specifically, we impose the following speed and rotation constraints:

$$v_{s_t} \leq v_{\max} \quad \forall t \tag{11}$$

$$|\phi_t - \phi_{t-1}| \leq \Delta t \cdot \mathcal{T}_r \quad \forall t \tag{12}$$

where v_{s_t} and ϕ_t are the speed and the orientation of the typical UAV at time step t, respectively. $v_{\max}$ is the maximum speed of the UAV, and $\mathcal{T}_r$ is the maximum rotation angle in unit time period.

4) Start and Destination Constraints: A UAV with a mission should fly from a given start point and arrive at a required destination, and hence we have

$$\mathbf{p}_0 = \mathbf{p}^S, \mathbf{p}_T = \mathbf{p}^D. \tag{13}$$

5) TDMA Constraint: Due to the utilization of TDMA, the UAV can communicate with at most one node at each time, and the constraint is given in (6).

Our goal is to maximize the collected data from all nodes subject to these constraints. Therefore, the optimization problem can be formulated as

$$\begin{aligned}(P1):\ &\underset{\{\mathbf{p}_t \quad \forall t\}}{\operatorname{argmax}} \sum_{t=0}^{T}\sum_{n=1}^{N} q_{nt}\Delta t R_{nt}\\ &\text{subject to } (6), (8), (9), (10), (11), (12), (13)\end{aligned}$$

where the subscript t in q_{nt} and R_{nt} is used to indicate the scheduling result and the information rate, respectively, at time step t.

III. Deep-Reinforcement-Learning-Based Path Planning

In this section, we first introduce the basic idea behind RL, then describe the RL formulation of decentralized path planning for data collection in a multi-UAV scenario, and finally explain the proposed D3QN path-planning algorithm in detail.

A. Reinforcement Learning

RL is a class of machine learning methods that can be utilized for solving sequential decision making problems with unknown state-transition dynamics [33], [34]. Typically, a sequential decision-making problem can be formulated as an MDP [35], which is described by the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ is the state-transition model, $\mathcal{R}$ is the reward function, and $\gamma \in [0, 1]$ is a discount factor that trades off the importance of the immediate and future rewards. More specifically, at each time step, an agent, in state $s_t \in \mathcal{S}$, chooses an action $a_t \in \mathcal{A}$, transitions to next state s_{t+1}, and receives reward $\mathcal{R}_t$ from the environment $\mathcal{E}$. The cumulative discounted reward $\mathcal{R}_t^C$ is defined as

$$\mathcal{R}_t^C \triangleq \sum_{\tau=0}^{\infty} \gamma^\tau \mathcal{R}_{t+\tau}. \tag{14}$$

The state–action-value function (also known as the Q-value) quantifies the expected return from a state–action pair (s, a), and can be expressed by the cumulative discounted reward

$$Q^\pi(s_t, a_t) \triangleq E_\pi\left[\mathcal{R}_t^C | s_t, a_t\right] \tag{15}$$

when policy π is followed. The Q-value satisfies the Bellman optimality equation [35]

$$Q^\pi(s_t, a_t) = E_\pi\left[r_t + \gamma \max_{a'} Q^\pi\left(s_{t+1}, a'\right) | s_t, a_t\right]. \tag{16}$$

The essential task of many RL algorithms is to seek the optimal policy that maximizes the expected cumulative discounted reward by solving

$$\pi^*(s_t) = \underset{a_t}{\operatorname{argmax}}\, Q(s_t, a_t). \tag{17}$$

QL is one of the most widely used algorithms for RL. In traditional QL, a table (referred to as the Q-table) is constructed, in which the component in row s and column a is the Q-value $Q(s, a)$. The QL update rule can be written as [36]

$$Q(s, a) \leftarrow Q(s, a) + \alpha\left[\mathcal{R} + \gamma \max_{a'} Q\left(s', a'\right) - Q(s, a)\right] \tag{18}$$

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

where α is a scalar step size and s' is the state in the next time step.

In problems with large state and action spaces, it becomes infeasible to use the Q-table. A typical approach is to convert the update problem of the Q-table into a function fitting problem, i.e., we can learn a parameterized value function $Q(s, a; \boldsymbol{\xi}) \approx Q(s, a)$ with parameters $\boldsymbol{\xi}$. When combined with deep learning, this leads to DQN. The operation of DQN consists of an online deep Q learning phase and an offline deep neural network (DNN) construction phase, which is used to learn the value function $Q(s, a; \boldsymbol{\xi})$. Generally, the parameter set $\boldsymbol{\xi}$ is optimized by minimizing the following loss function [37]:

$$\mathcal{L}_t(\boldsymbol{\xi}_t) = E\Big[\big(y_t - Q(s_t, a_t; \boldsymbol{\xi}_t)\big)^2\Big] \tag{19}$$

where $y_t = \mathcal{R}_t + \gamma \max_{a'} Q(s_{t+1}, a'; \boldsymbol{\xi}_t^-)$ is the target, and $\boldsymbol{\xi}_t^-$ is copied at certain steps from $\boldsymbol{\xi}_t$.

The maximization operator in standard QL and DQN uses the same values both to select and to evaluate an action, which increases the probability to select overestimated values and results in overoptimistic value estimates. DDQN can be used to mitigate the above problem by using the following target [38]:

$$y_t = \mathcal{R}_t + \gamma Q\Big(s_{t+1}, \operatorname*{argmax}_{a'} Q\big(s_{t+1}, a'; \boldsymbol{\xi}_t\big); \boldsymbol{\xi}_t^-\Big) \tag{20}$$

where $\boldsymbol{\xi}_t$ is used for action selection, and $\boldsymbol{\xi}_t^-$ is used to evaluate the value of the policy, and $\boldsymbol{\xi}_t^-$ can be updated symmetrically by switching the roles of $\boldsymbol{\xi}_t$ and $\boldsymbol{\xi}_t^-$.

Furthermore, considering that Q-value measures how beneficial a particular action a is when taken in state s, the dueling architecture is introduced to obtain a value $V(s)$ and an advantage $A(s, a) = Q(s, a) - V(s)$ [39]. The value $V(s)$ is to measure how good it is to be in a particular state s. The advantage $A(s, a)$ describes the advantage of the action a compared with other possible actions while in state s [40]. Therefore, the difference in dueling DQN, compared with DQN, is that the last layer of the DQN is split into two separate layers, $\boldsymbol{\xi}^V$ and $\boldsymbol{\xi}^A$. $\boldsymbol{\xi}^V$ is used to obtain the value $V(s; \boldsymbol{\xi}, \boldsymbol{\xi}^V)$, and the output of $\boldsymbol{\xi}^A$ is the advantage for each action $A(s, a; \boldsymbol{\xi}, \boldsymbol{\xi}^A)$. The Q-value in dueling DQN can be expressed as [39]

$$\begin{aligned} &Q\Big(s, a; \boldsymbol{\xi}, \boldsymbol{\xi}^V, \boldsymbol{\xi}^A\Big) \\ &\quad = V\big(s; \boldsymbol{\xi}, \boldsymbol{\xi}^V\big) + A\Big(s, a; \boldsymbol{\xi}, \boldsymbol{\xi}^A\Big) - \frac{1}{|\mathcal{A}|}\sum_{a'} A\Big(s, a'; \boldsymbol{\xi}, \boldsymbol{\xi}^A\Big). \end{aligned} \tag{21}$$

The general structure of dueling DQN is displayed in Fig. 3. Dueling DDQN (D3QN) is a combination of dueling DQN and DDQN. The learning strategy within D3QN is more rewarding, and we in this article utilize D3QN to achieve path planning for data collection in multi-UAV scenarios.

B. Reinforcement Learning Formulation

Considering the objective function in (P1) and the constraints in (6)–(13), we can translate the considered problem into an MDP, and the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{R} \rangle$ is explained in detail as follows.

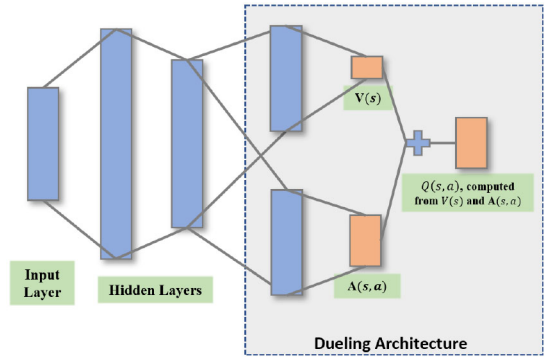


Fig. 3. Structure of the dueling deep Q-network.

1) State: In multi-UAV scenarios, the typical UAV is able to obtain the following information:

1) Its own full information vector $\mathbf{s}_t$ at time step t, where $\mathbf{s}_t = [p_{x_t}, p_{y_t}, H_V, v_{x_t}, v_{y_t}, r, p_{gx}, p_{gy}, H_V, v_{\max}, \theta_t]$.
2) The observable information vector of other UAVs in $\mathbb{O}$, which is the sensing region of the typical UAV. The total number of observed other UAVs at time step t is $J_t^o \geq 0$, and the joint information vector can be expressed as $\mathbf{s}_t^o = [\mathbf{s}_{jt}^o : j \in \{1, 2, \ldots, J_t^o\}]$, where $\mathbf{s}_{jt}^o = [p_{x_{jt}}, p_{y_{jt}}, H_V, v_{x_{jt}}, v_{y_{jt}}, r]$.
3) The location information $\mathbf{p}_{nt}$ of each IoT node, the amount of remaining data D_{nt}^L at each node,[1] and the received signal power P_{nt}^r from each node. The joint vector is denoted by $\mathbf{s}_t^n = [\mathbf{s}_{nt}^n : n \in \{1, \ldots, N\}]$, where $\mathbf{s}_{nt}^n = [\mathbf{p}_n, D_{nt}^L, P_{nt}^r]$.
4) The available time left for the given mission, s_{tt}.

Note that there may be other UAVs outside the typical UAV's sensing region $\mathbb{O}$, and thus they are not observed. However, since these UAVs are far away from the typical UAV, their existence do not influence the typical UAV's action, and correspondingly can be neglected. Therefore, all the observed information represents the state of the typical UAV's surrounding environment, which can be written as $\mathbf{s}_t^{jn} = [\mathbf{s}_t, \mathbf{s}_t^o, \mathbf{s}_t^n, s_{tt}]\ \forall t$.

To aid the UAV in interpreting the large state space and obtaining more information from the state, we implement the state parameterization process, which consists of the following three steps.

1) *Coordinate Transition:* The policy should not be influenced by the choice of coordinates. Therefore, we parameterize the position information into agent-centric coordinates, in which the current location of the typical UAV is regarded as the origin, and the x-axis is pointing toward the UAVs destination. The coordinate rotation angle can be expressed as

$$\theta_t^r = \arctan\left(\frac{p_{gy} - p_{y_t}}{p_{gx} - p_{x_t}}\right).$$

Then, the transitions from global coordinates into UAV-centric coordinates can be expressed as

$$\tilde{p}_{x_{jt}} = \big(p_{x_{jt}} - p_{x_t}\big)\cos\big(\theta_t^r\big) + \big(p_{y_{jt}} - p_{y_t}\big)\sin\big(\theta_t^r\big)$$

[1] D_{nt}^L can be obtained from D_{nt-1}^L, P_{nt}^r, and the scheduling strategy.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

$$\begin{aligned}\tilde{p}_{y_{jt}} &= \left(p_{y_{jt}} - p_{y_t}\right)\cos\left(\theta_t^r\right) - \left(p_{x_{jt}} - p_{x_t}\right)\sin\left(\theta_t^r\right)\\ \tilde{v}_{x_{jt}} &= v_{x_{jt}}\cos\left(\theta_t^r\right) + v_{y_{jt}}\sin\left(\theta_t^r\right)\\ \tilde{v}_{y_{jt}} &= v_{y_{jt}}\cos\left(\theta_t^r\right) - v_{x_{jt}}\sin\left(\theta_t^r\right)\\ \tilde{\theta}_t &= \theta_t - \theta_t^r.\end{aligned}$$

After the coordinate transition, we have the rotated information vectors as follows:

$$\begin{aligned}\tilde{\mathbf{s}}_t &= \left[\tilde{v}_{x_t}, \tilde{v}_{y_t}, r, \tilde{p}_{gx_t}, \tilde{p}_{gy_t}, v_{\max}, \tilde{\theta}_t\right]\\ \tilde{\mathbf{s}}_{jt}^o &= \left[\tilde{p}_{x_{jt}}, \tilde{p}_{y_{jt}}, \tilde{v}_{x_{jt}}, \tilde{v}_{y_{jt}}, r_j\right] \text{ for } j \in \left\{1, \ldots, J_t^o\right\}\\ \tilde{\mathbf{s}}_{nt}^n &= \left[\tilde{p}_{x_{nt}}, \tilde{p}_{y_{nt}}, D_{nt}^L, P_{nt}^r\right] \text{ for } n \in \{1, \ldots, N\}.\end{aligned}$$

2) *Processing of the State Components:* The distance, relative direction, and how strong the received signal power is from each node have significant influence on the typical UAV's decision making. Therefore, the components of the state are processed to provide more information, and we have

$$\begin{aligned}d_{jt} &= \sqrt{p_{x_{jt}}^2 + p_{y_{jt}}^2}\\ a_{jt} &= \arctan\left(\frac{p_{y_{jt}}}{p_{x_{jt}}}\right)\\ \mathbb{1}_{nt} &= \begin{cases}1, & \text{if } \frac{P_{nt}^r}{\mathcal{N}_s} \geq \mathcal{T}_s\\ 0, & \text{otherwise}\end{cases}\end{aligned}$$

where $\mathbb{1}_{nt}$ is the indicator function, which is used to indicate whether the SNR is larger than threshold $\mathcal{T}_s$ if the typical UAV is connected with the nth node.

3) *State Size Management:* Since the typical UAV can only observe other UAVs inside its sensing region, $\mathbb{O}$, the total number of observed other UAVs, J_t^o, may vary over time. However, the size of the state needs to be fixed to be the input of the DNN. Thus, we only consider the information vectors of the nearest J^c (which is a fixed positive integer) other UAVs. In addition, the number of nodes in the environment may be different for different missions. Thus, we only consider the information vectors of the nearest N^c active nodes. If $J_t^o < J^c$ or the number of active nodes is smaller than N^c, we do zero padding.

After these three steps, we have the parameterized state as

$$\tilde{\mathbf{s}}_t^{jn} = \left[\tilde{\mathbf{s}}_t, \left[\tilde{\mathbf{s}}_{jt}^o \quad \forall j\right], \left[\tilde{\mathbf{s}}_{nt}^n \quad \forall n\right], s_{tt}\right] \tag{22}$$

where

$$\begin{aligned}\tilde{\mathbf{s}}_t &= \left[\tilde{v}_{x_t}, \tilde{v}_{y_t}, \tilde{p}_{gx_t}, \tilde{p}_{gy_t}, d_{g_t}, a_{g_t}, r, v_{\max}, \tilde{\theta}_t\right]\\ \tilde{\mathbf{s}}_{jt}^o &= \left[\tilde{p}_{x_{jt}}, \tilde{p}_{y_{jt}}, \tilde{v}_{x_{jt}}, \tilde{v}_{y_{jt}}, d_{jt}, a_{jt}, r_j\right]\\ &\quad \text{for } j \in \left\{1, 2, \ldots, \min\left(J_t^o, J^c\right)\right\}\\ \tilde{\mathbf{s}}_{nt}^n &= \left[\tilde{p}_{x_{nt}}, \tilde{p}_{y_{nt}}, d_{nt}, a_{nt}, D_{nt}^L, P_{nt}^r, \mathbb{1}_{nt}\right] \text{ for } n \in \left\{1, \ldots, N^c\right\}.\end{aligned}$$

2) Action: In an ideal setting, the agent can travel in any direction at any time. However, in practice, kinematic constraints in (11) and (12) restrict the agent's movement and should be taken into account. Given these constraints, permissible velocities $[v_s, \phi_r]$ are sampled to built a velocity-set, where v_s is the permissible speed, and the ϕ_r is the permissible rotation angle. The action a is the index of each velocity in the velocity-set.

3) Reward: The reward can be designed according to the objective function and the constraints, and the design plays an important role on the learning speed and quality. The reward function of this path-planning problem for data collection in the multi-UAV scenario can be expressed as

$$\mathcal{R}_t = \mathcal{R}_{dt} + \mathcal{R}_{ct} + \mathcal{R}_{ot} + \mathcal{R}_{tt} + \mathcal{R}_{gt} + \mathcal{R}_{st}. \tag{23}$$

The first term $\mathcal{R}_{dt}$ is related to the data collected from the nodes during next time duration Δt. This reward term is used to encourage the UAV to collect data from the IoT nodes, and can be expressed as

$$\mathcal{R}_{dt} = \alpha_1 \times \left(\sum_{n=1}^{N} D_{nt}^L - \sum_{n=1}^{N} D_{nt+1}^L\right). \tag{24}$$

$\mathcal{R}_{ct}$ is the term introduced to penalize collision with other UAVs and encourage the typical UAV stay further away from the other UAVs. This term can be formulated as

$$\mathcal{R}_{ct} = \begin{cases}-\alpha_2, & \text{if } d_{t_{\min}} \leq r + r_j\\ -\alpha_2 \times \left(1 - \frac{d_{t_{\min}} - r - r_j}{d_b}\right), & \text{if } r + r_j < d_{t_{\min}} \leq d_b + r + r_j\\ 0, & \text{otherwise}\end{cases} \tag{25}$$

where $d_{t_{\min}}$ is the minimum distance from the typical UAV to other UAVs during next time duration Δt, and d_b is a constant that denotes the distance buffer. $\mathcal{R}_{ot}$ is to penalize the collision with the fixed obstacles or entering into no-fly zones, and can be expressed as

$$\mathcal{R}_{ot} = \begin{cases}-\alpha_3, & \text{if } \mathbf{p}_{t+1} \in \mathbb{N}\\ 0, & \text{otherwise.}\end{cases} \tag{26}$$

$\mathcal{R}_{tt}$ is the reward related to mission completion deadline constraint, and it encourages the UAVs to arrive at their destinations within the allowed duration of time, and can be formulated as

$$\mathcal{R}_{tt} = \begin{cases}\alpha_4 \times \left(s_{tt+1} - T_{gt+1}^{\min}\right), & \text{if } s_{tt+1} < T_{gt+1}^{\min}\\ 0, & \text{otherwise}\end{cases} \tag{27}$$

where s_{tt+1} is the available time left for the given mission, $T_{gt+1}^{\min} = d_{gt+1}/v_{\max}$ is the minimum time duration needed to reach destination at time step $t+1$, and d_{gt+1} is the distance to destination at time step $t+1$. R_{gt} is the reward given for arriving the destination, and

$$\mathcal{R}_{gt} = \begin{cases}\alpha_5, & \text{if } \mathbf{p}_{t+1} = \mathbf{p}^D\\ 0, & \text{otherwise.}\end{cases} \tag{28}$$

The last term $\mathcal{R}_{st} = -\alpha_6$ is a step penalty for each movement, and it is used to encourage fast arrival. Note that $\alpha_{1\sim6}$ are positive constants, and can be varied to adjust the weight or emphasis of each reward term to adapt to different mission priorities.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: D3QN Path-Planning Algorithm for Data Collection

Input: $\mathcal{T}_s$, $\mathcal{T}_t$, $v_{\max}$, $\mathcal{T}_r$

1 Initialize replay memory $\mathcal{D}$
2 Initialize evaluation network $\boldsymbol{\xi}$ (including $\boldsymbol{\xi}^V$ and $\boldsymbol{\xi}^A$)
3 Initialize target network $\boldsymbol{\xi}^-$ (including $\boldsymbol{\xi}^{V-}$ and $\boldsymbol{\xi}^{A-}$) by copping from $\boldsymbol{\xi}$
4 $\mathcal{A} \leftarrow$ sampleActionSpact($v_{\max}, \mathcal{T}_r$)
5 $\mu, \sigma \leftarrow$ getMeanStandardDeviation()
6 **for** *episode = 0: total episode* N_e **do**
7 $\quad\mathcal{E} \leftarrow$ resetEnvironment()
8 $\quad$**while** *not done* **do**
9 $\quad\quad\mathbf{s}_t^{jn} \leftarrow$ observeEnvironment($\mathcal{E}$)
10 $\quad\quad\widetilde{\mathbf{s}}_t^{jn} \leftarrow$ parameterizeState($\mathbf{s}_t^{jn}$)
11 $\quad\quad\widetilde{\mathbf{s}}_t^{jn} \leftarrow$ normalizeState($\widetilde{\mathbf{s}}_t^{jn}, \mu, \sigma$)
12 $\quad\quad c \leftarrow$ randomSample(Uniform (0,1))
13 $\quad\quad$**if** $c \leq \epsilon$ **then**
14 $\quad\quad\quad\mathbf{a}_t \leftarrow$ randomSample($\mathcal{A}$)
15 $\quad\quad$**else**
16 $\quad\quad\quad\mathbf{a}_t \leftarrow \underset{\mathbf{a}' \in \mathcal{A}}{\operatorname{argmax}}\, Q(\widetilde{\mathbf{s}}_t^{jn}, \mathbf{a}'; \boldsymbol{\xi})$
17 $\quad\quad\mathcal{R}_t, \mathbf{s}_{t+1} \leftarrow$ executeAction($\mathbf{a}_t$)
18 $\quad\quad\widetilde{\mathbf{s}}_{t+1}^{jn} \leftarrow$ parameterizeState($\mathbf{s}_{t+1}^{jn}$)
19 $\quad\quad\widetilde{\mathbf{s}}_{t+1}^{jn} \leftarrow$ normalizeState($\widetilde{\mathbf{s}}_{t+1}^{jn}, \mu, \sigma$)
20 $\quad\quad$Uptate D with tuple $(\widetilde{\mathbf{s}}_t^{jn}, \mathbf{a}_t, \mathcal{R}_t, \widetilde{\mathbf{s}}_{t+1}^{jn})$
21 $\quad\quad$Sample a minibatch of N_b tuples $(\mathbf{s}, \mathbf{a}, \mathcal{R}, \mathbf{s}') \sim$ Uniform(D)
22 $\quad\quad$**for** *each tuple j* **do**
23 $\quad\quad\quad$Calculate target $y_j =$
24 $\quad\quad\quad\quad\begin{cases} \mathcal{R}, & \text{if } \mathbf{s}' \text{ is terminal,} \\ \mathcal{R} + \gamma Q(\mathbf{s}', \underset{\mathbf{a}'}{\operatorname{argmax}}\, Q(\mathbf{s}', \mathbf{a}'; \boldsymbol{\xi}); \boldsymbol{\xi}^-), & \text{o.w.} \end{cases}$
25 $\quad\quad$Do a gradient descent step with loss $E[(y_j - Q(\mathbf{s}, \mathbf{a}; \boldsymbol{\xi}))^2]$
26 $\quad\quad$Update $\boldsymbol{\xi}^- \leftarrow \boldsymbol{\xi}$ every N_r steps
27 **return** $\boldsymbol{\xi}$

C. D3QN Path-Planning Algorithm for Data Collection

The main algorithm is summarized in Algorithm 1, where the input consists of the parameters of the constraints and the output is the policy, i.e., a well-trained DNN $\boldsymbol{\xi}$. In the training phase, we first initialize the replay memory, the evaluation network parameters and the target network parameters (lines 1–3). We also build an action space $\mathcal{A}$ based on the kinematic constraints (line 4). Before training, we randomly generated the following: the locations of typical UAV, the IoT nodes, and other UAVs; the amount of data to be collected from each node; and the velocities of all UAVs; for 10^6 times. We then find the state vector of each generation, and calculate the mean vector μ and standard deviation vector σ of the state (line 5). In each training episode, the typical UAV navigates around other UAVs and obstacles to arrive at its destination, while collecting data from ground IoT nodes. Particularly, at the beginning of each episode, the environment and the UAV's mission are reset (line 7), and the parameters that are reset are as follows.

1) The starting points and destinations of the typical UAV.
2) The number of IoT nodes.
3) The locations of IoT nodes.
4) The amount of data to be collected from each node.
5) The number of other UAVs.
6) The starting points and destinations of other UAVs.

Then, at each time step t, the typical UAV observes the environment, obtains the observation vector $\mathbf{s}_t^{jn}$ (line 9), parameterizes the vector into $\widetilde{\mathbf{s}}_t^{jn}$ following the three steps introduced in the previous section, and standardizes the state by $(\widetilde{\mathbf{s}}_t^{jn} - \mu)/\sigma$ (line 11). Using an ϵ-greedy policy, the typical UAV selects a random action with probability ϵ from $\mathcal{A}$ (line 14), or follows the policy greedily otherwise (line 16). The Q-value can be obtained using the evaluation network $\boldsymbol{\xi}$ according to (21). After executing the chosen action, the typical UAV receives reward $\mathcal{R}_t$ from the environment according to (23), and it observes the new state $\mathbf{s}_{t+1}^{jn}$ from the updated environment (line 17). Then, the replay memory is updated with transition tuple $(\widetilde{\mathbf{s}}_t^{jn}, \mathbf{a}_t, \mathcal{R}_t, \widetilde{\mathbf{s}}_{t+1}^{jn})$ (line 20). To train the evaluation network $\boldsymbol{\xi}$, a minibatch of N_b tuples can be randomly sampled from replay memory (line 21). Then, $\boldsymbol{\xi}$ is updated by stochastic gradient descent (back-propagation) on the sampled minibatch (lines 22–25), and the target network parameters $\boldsymbol{\xi}^-$ are updated from $\boldsymbol{\xi}$ for every N_r steps (line 26). The episode ends when the UAV arrives at its destination or exceeds its mission deadline. Lines 7–26 can be repeated for N_e episodes. After training, we obtain a policy, i.e., a well-trained DNN $\boldsymbol{\xi}$, with which the UAV can perform real-time navigation.

IV. Numerical Results

In this section, we present the numerical and simulation results to evaluate the performance of the proposed algorithm. The considered performance metrics are the following: 1) SR, and a successful trajectory means that the UAV arrives at its destination within mission completion deadline without collisions; 2) DR, which is the percentage of collected data within successful missions; 3) data collection and SR (DSR), which is the product of data collection percentage and SR; and 4) CR, and a collision event occurs when the typical UAV collides with any of the other UAVs in the environment. In the illustration of real-time navigation scenarios, the departure and landing areas of the typical UAV are displayed by blue and green areas, respectively. The no-fly zones (obstacles) are presented in gray areas. The IoT nodes are marked by green triangles. The trajectories of the typical UAV are presented by navy lines with dots, and the trajectories of other UAVs in the environment are depicted in red lines with dots. The destination of each mission is marked by a navy cross in the landing area. Note that since the UAVs may arrive at the same location at different times, they do not necessarily collide even if their trajectories intersect. In the simulations, other UAVs use optimal reciprocal collision avoidance (ORCA) [41] in choosing actions and determining their trajectories.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

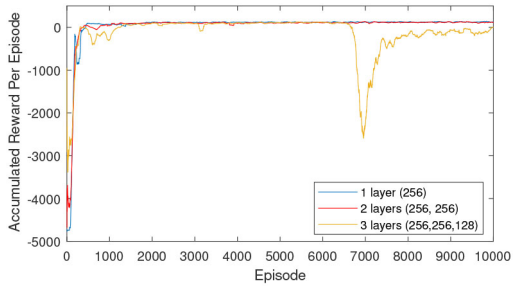


Fig. 4. Accumulated reward per episode in training with different DNN structures.

TABLE II
SR, DR, DSR, AND CR PERFORMANCE WHEN DIFFERENT NUMBER OF NODES (N) NEED TO UPLOAD DATA, AND $D_0^L \in [1, 3]$, $J \in [2, 10]$

	N=5	N=6	N=7	N=8	N=9	N=10
SR(%) ($\mathcal{T}_t = 100$s)	95.5	94.4	93.8	92.9	92.5	91.5
DR(%)	99.9	99.8	99.8	99.8	99.8	99.8
DSR(%)	95.4	94.21	93.61	95.45	92.71	91.31
CR(%)	3.3	3.4	3.9	3.9	3.9	3.7
SR(%) ($\mathcal{T}_t = 200$s)	96.7	96.2	95.8	95.8	95.8	95.9

A. Environment Setting and Hyperparameters

Since the agents fly at the same altitude, the area of interest becomes 2-D. In the simulations, the UAV flies at height $H_V =$ 50m. The size of the area of interest is scaled to a 100 $\times$ 100 region. The IoT nodes have the same transmit power of $P = 1$ dBm. Noise power is $\mathcal{N}_s = 10^{-6}$. SNR threshold is set at $\mathcal{T}_s = -5$ dB (unless stated otherwise). Mission completion deadline is $\mathcal{T}_t$ = 100 s (unless stated otherwise). Kinematic constraints are $v_{\max}$ = 5 in the scaled form and $\mathcal{T}_r = \pi/3$. The radius of the UAV's sensing region is 10, N^c is 5, and J^c is set to be 2.

Fig. 4 shows the accumulated reward per episode in training with different DNN structures. We can observe from the figure that one-layer and two-layer structures can achieve similar convergence speeds and reward performance, while the reward from DNN with a three-layer structure drops after 6500 episodes and then grows back. Therefore, we choose to use two-layer DNN of size (256, 256). ReLU function is used as the activation function, Batch-normalization is used for each layer, and the Adam optimizer is used to update parameters with learning rate 0.0003. The batch size is 256, and the regularization parameter is 0.0001. The exploration parameter ϵ decays linearly from 0.5 to 0.1. The replay memory capacity is 1 000 000.

B. Training

The total number of training episodes is 10 000. In each episode, the following system parameters are randomly chosen from the corresponding regions and sets of values.

1) The starting points and destinations of the typical UAV $\mathbf{p}^S \in \mathbb{S}$ and $\mathbf{p}^D \in \mathbb{D}$.
2) The number of IoT nodes $N \in [5, 10]$.
3) The locations of ground IoT nodes $\mathbf{p}_n \in \mathbb{C}$.
4) The amount of data to be collected from each node $D_0^L \in [1, 3]$ data units.
5) The number of other UAVs $J \in [2, 10]$.
6) The starting points and destinations of other UAVs $\mathbf{p}_j \in \mathbb{C}$.

Fig. 5(a)–(c) shows the accumulated reward per episode, SR per 100 episodes, and DR per episode, respectively, during different training sessions with two-layer DNN. These figures show that in different training sessions, similar performances (in terms of the convergence speed and attained levels of performance metrics) are achieved, indicating the stability of the proposed algorithm. In addition, the SR eventually converges to around 92.5%, and the DR reaches 99.5%, even with the exploration strategy (i.e., with $\epsilon_{\min} = 0.1$). The simulation is implemented on a Windows 10 with Intel Core i7-8753h CPU. In total, the training algorithm converges after approximately 6000 episodes for this multi-UAV scenario with highly varying parameters.

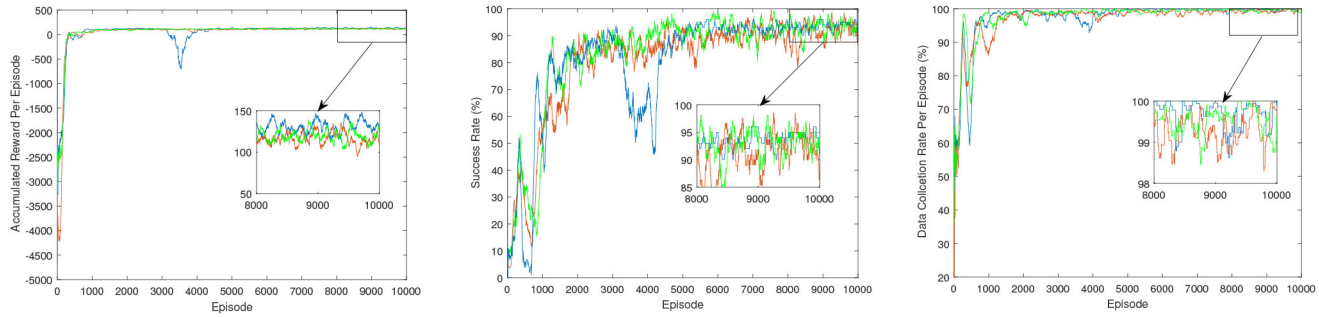
C. Testing of Navigation in Different Scenarios

With the learned policy, the UAV can perform real-time navigation in different scenarios considering the various parameters described in the previous section. We note that all the testing in this section uses the same policy, i.e., the policy can adapt to various missions and scenarios without further training.

1) Different Number of IoT Nodes: Fig. 6 displays illustrations of navigation in different scenarios in which $N \in [8, 10]$ IoT nodes are randomly located, $D_0^L = 1$ data unit, and $J \in [2, 10]$. Fig. 6 shows that for different numbers and locations of IoT nodes, different numbers and locations of other UAVs, and different start points and destinations, the UAV can adjust its trajectory to collect data from distributed IoT nodes with the trained policy. Table II provides the SR, DR, DSR, and CR performances in testing for different number of nodes from which data needs to be collected. The performances are averaged over 5000 random realizations (in each of which, we have $D_0^L \in [1, 3]$, $J \in [2, 10]$, and all UAVs have random starting points and destinations). Overall, Table II shows that the proposed algorithm can achieve above 91% SR (for tight mission completion deadline constraint of $\mathcal{T}_t$) and over 99.8% DR when $N \in [5, 10]$. With the increasing number of nodes, generally the UAV needs to plan a longer trajectory to get close to each node to achieve reliable communication (i.e., to satisfy $S_n \geq \mathcal{T}_n$), leading to relatively longer flight durations and higher risk for collision. Therefore, when N increases, SR decreases due to the increase in CR and the increase in flight duration. In addition, when we compare row 1 (smaller $\mathcal{T}_t$) and row 5 (larger $\mathcal{T}_t$), we notice that higher SR is obtained if the mission completion deadline is relaxed.

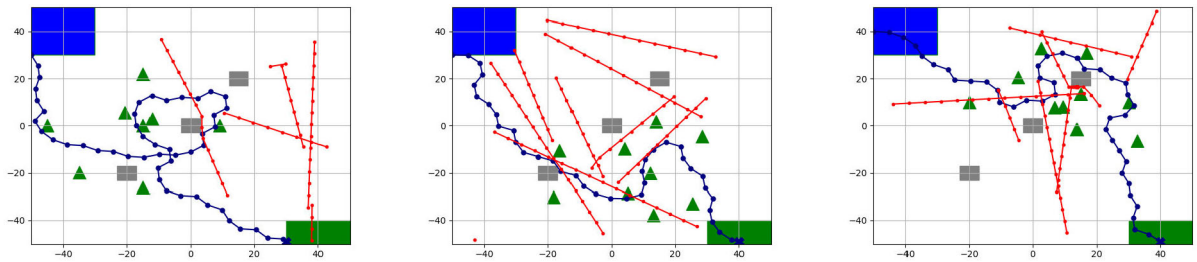
2) Different Amount of Data at Each Node: Fig. 7 depicts illustrations of navigation in different scenarios in which $D_0^L \in [1, 3]$ data units need to be collected from each node. To display the influence of the amount of data to be collected, we fix the number of nodes as $N = 7$ and the number of other UAVs as $J = 2$ in the illustrations. Due to the different

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.



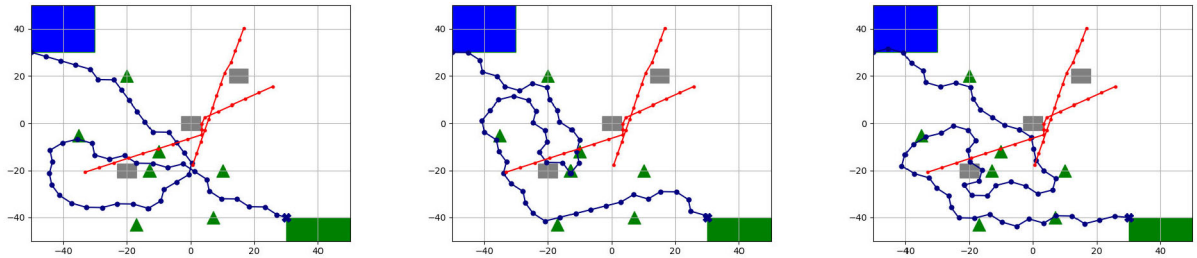
(a) (b) (c)

Fig. 5. Accumulated reward per episode, average SR per 100 episodes and DR per episode for different training cases. (a) Reward. (b) Success rate. (c) Data collection rate.



(a) (b) (c)

Fig. 6. Illustrations of navigation in different scenarios that $N \in \{8, 9, 10\}$ nodes are randomly located, and $D_0^L = 1$ data unit, $J \in [2, 10]$. (a) Eight nodes. (b) Nine nodes. (c) Ten nodes.



(a) (b) (c)

Fig. 7. Illustrations of navigation in different scenarios that $D_0^L \in \{1, 2, 3\}$ data units need to be collected at each node, and $N = 7$, $J = 2$. (a) 1 data unit. (b) 2 data units. (c) 3 data units.

TABLE III
SR, DR, AND DSR PERFORMANCE WHEN DIFFERENT AMOUNT OF DATA D_0^L NEEDS TO BE COLLECTED FROM EACH NODE, AND $N \in [5, 10]$, $J \in [2, 10]$

	D_0^L=1	D_0^L=2	D_0^L=3
SR(%)	94.9	93.7	92.8
DR(%)	99.8	99.8	99.7
DSR(%)	94.71	93.51	92.52
CR(%)	2.8	3.7	3.9

amount of data to be collected at each node, the UAV needs to fly around each node over a different duration of time to complete the data collection, leading to different trajectories. Table III provides the SR, DR, DSR, and CR performances in testing, when the values of D_0^L are different. The rates are averaged over 5000 random realizations (in each of which $N \in [5, 10]$, $J \in [2, 10]$, and all UAVs have random starting points and destinations). We observe similar performance levels as in Table II, i.e., when there is more data to collect, the UAV needs a longer trajectory and a longer time period to complete the mission, leading to higher CR and lower SR.

3) Different Number of Other UAVs: Fig. 8 presents illustrations of navigation in scenarios with different number of other UAVs $J \in \{2, 10, 20\}$. Again, to display the impact of different values of J, we fix $N = 6$ and $D_0^L = 1$ in the illustrations. From Fig. 8, we can observe that due to the different locations and numbers of other UAVs, the typical UAV makes decisions to avoid collisions, leading to different trajectories. Table IV presents the CR, SR, and DR performances as J varies, considering three different hyperparameter settings: *setting* 1 (S1) $\alpha_2 = 10, d_b = 0.2$, and $T_t = 100$ s; *setting* 2 (S2) $\alpha_2 = 30, d_b = 1$, and $T_t = 200$ s; and *setting* 3 (S3) $\alpha_2 = 50, d_b = 10$, and $T_t = 200$ s. The rates are averaged over 5000 random realizations (with $N \in [5, 10]$, $D_0^L \in [1, 3]$, and all UAVs having random starting points and destinations).

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

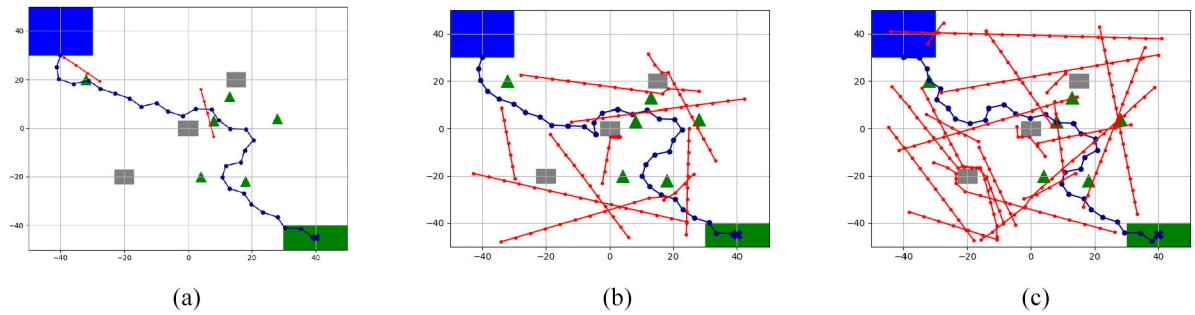


Fig. 8. Illustrations of navigation in different scenarios involving $J \in \{2, 10, 20\}$ other UAVs, and $N = 6$, $D_0^L = 1$ data unit. (a) 2 other UAVs. (b) Ten other UAVs. (c) Twenty other UAVs.

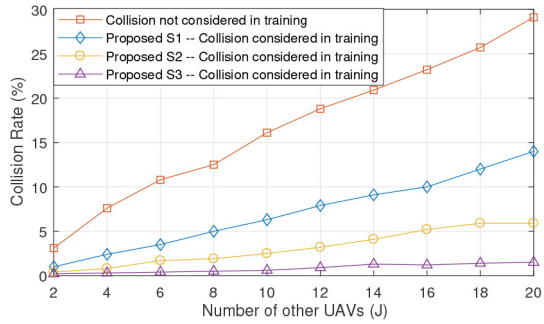


Fig. 9. CR comparison in testing between two methods: 1) proposed approach with setting 1 and setting 2, in which the collision avoidance in the presence of arbitrary number of other UAVs is taken into account in training and 2) approach in which collision with other UAVs is not considered in training.

TABLE IV
CR, SR, AND DR PERFORMANCE WHEN DIFFERENT NUMBER OF OTHER UAVS J EXIST, AND $N \in [5, 10]$, $D_0^L \in [1, 3]$

		J=2	J=4	J=6	J=8	J=10	J=12
S1	**CR(%)**	**1**	**2.4**	**3.5**	**5**	**6.3**	**7.9**
	SR(%)	96	94.6	93.9	92.2	91.5	90.7
	DR(%)	99.8	99.8	99.8	99.8	99.8	99.9
S2	**CR(%)**	**0.4**	**0.8**	**1.7**	**1.9**	**2.5**	**3.2**
	SR(%)	94.4	94	92.7	93.2	92.6	91.8
	DR(%)	99.8	99.8	99.8	99.8	99.8	99.8
S3	**CR(%)**	**0.1**	**0.2**	**0.2**	**0.3**	**0.4**	**0.4**
	SR(%)	99	98.8	98.8	98.8	98.4	98.2
	DR(%)	99.7	99.7	99.7	99.7	99.8	99.7

From the table, we note that with increasing number of other UAVs, the CR grows due to higher risk of collision. When CR performances in the three settings are compared, we can observe that if the mission completion deadline is loosened, the distance buffer, d_b, between two UAVs is increased and the penalty for collision, α_2, is increased, CR can be reduced. Specifically, with *setting 3*, CR level of 0.4% can be achieved for a crowded scenario with $J = 12$. This observation indicates that the hyperparameters can be tuned to adapt to different mission priorities, and, for instance, substantially reduce CR.

To show the importance of considering collision avoidance in the presence of an arbitrary number of other UAVs in the environment, Fig. 9 compares the CR levels achieved with the proposed algorithm (which considers collision avoidance in training) and the CR levels attained with the learning algorithm that does not address collision avoidance with other UAVs in training. The figure shows us that if collision avoidance is not considered, we have significantly higher CR than those achieved with the proposed algorithm, and also the gap between two methods becomes larger with increasing number of other UAVs. For example, when $J = 20$, if collision avoidance is not considered, the CR rises up to 29.1%, while it is only 0.6% with the proposed algorithm. These observations indicate that the algorithms, which do not consider the existence of other UAVs or only consider fixed number of known UAVs, may lead to high collision risk in crowded scenarios, and the proposed algorithm greatly reduces that risk.

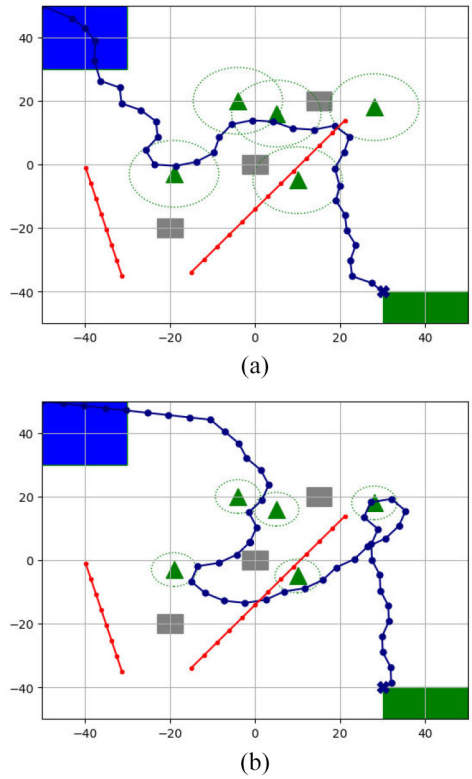


Fig. 10. Illustrations of navigation when SNR threshold $\mathcal{T}_s$ is different, and $N = 5$, $D_0^L = 1$ data unit, $J = 2$.

D. Impact of the SNR Threshold

Fig. 10 displays the influence of the values of the SNR threshold $\mathcal{T}_s$ on the UAV trajectories. Note that the UAV is able to achieve reliable communication with an IoT node only

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

TABLE V
CR PERFORMANCE WHEN THE OBSERVATIONS ARE NOISY AND $J = 20$

	$\mathbf{p}_j + \mathbf{n}$ u=1	$\mathbf{p}_j + \mathbf{n}$ u=3	$\mathbf{p}_j + \mathbf{n}$ u=5	No noise
CR(%)	0.8	1.1	1.3	0.6
	$\mathbf{v}_j + \mathbf{n}$ u=1	$\mathbf{v}_j + \mathbf{n}$ u=3	$\mathbf{v}_j + \mathbf{n}$ u=5	
CR(%)	0.6	0.8	1	
	$\mathbf{p}_j + \mathbf{n}$ $\mathbf{v}_j + \mathbf{n}$ u=1	$\mathbf{p}_j + \mathbf{n}$ $\mathbf{v}_j + \mathbf{n}$ u=3	$\mathbf{p}_j + \mathbf{n}$ $\mathbf{v}_j + \mathbf{n}$ u=5	
CR(%)	0.8	1.2	1.6	

if the UAV experiences an SNR that exceed the threshold, i.e., $S \geq \mathcal{T}_s$ for that connection. Since SNR is a function of the distance between the UAV and a node, $\mathcal{T}_s$ can be converted to a distance threshold. Therefore, different values of $\mathcal{T}_s$ lead to different distance requirements for reliable connection. The green dashed circles in Fig. 10 approximately indicate the area inside which we have $S \geq \mathcal{T}_s$. As shown in Fig. 10, the UAV has to reach each circle in order to collect data from the corresponding node. We observe that with higher $\mathcal{T}_s$ [Fig. 10(b) compared with Fig. 10(a)], the UAV needs to reach closer to each node for data collection, leading to different trajectories for the same mission.

E. Impact of Noisy Observations

The typical UAV avoids collisions mainly depending on the observations on nearby other UAVs. To investigate the robustness of the proposed algorithm, we perform simulations with noisy observations, i.e., adding noise to the observable information of other UAVs, $\mathbf{s}_j^o$. It is assumed that each noise component is uniformly distributed, i.e., $\mathbf{n} = [n_x, n_y]$ and $n \sim U(-u, u)$. The noise is added to the position information, i.e., $\mathbf{p}_j^n = [p_{jx} + n_x, p_{jy} + n_y]$, or the velocity information, i.e., $\mathbf{v}_j^n = [v_{jx} + n_x, v_{jy} + n_y]$, or both. Table V shows the CR performance when the observations are noisy. The results are obtained with setting-3 where $\alpha_2 = 50$ and $d_b = 10$. The table shows that noise has influence on the CR performance, since the typical UAV does not have the accurate information and correspondingly can with higher probability choose incorrect actions. However, even through noise with $u = 5$ is added to both position and velocity information vectors, CR is at most 1.6%, which is still small. Therefore, the proposed algorithm has high tolerance to noisy observations.

F. Comparison With Other Algorithms

Table VI presents the performances of different deep RL algorithms, i.e., D3QN, Dueling DQN, DDQN, and DQN, for data collection in multi-UAV scenarios. It can be clearly observed that D3QN has better performance than the other three, in terms of SR, DR, SDR, and CR, although the performances are not substantially different. It is worth noting that we have observed in training sessions that the reward of Dueling DQN, DDQN, and DQN may drop after a certain number of episode, while D3QN does not have this issue, indicating that D3QN is more stable than the other three algorithms when solving the considered problem.

TABLE VI
SR, AND DR, DSR, AND CR PERFORMANCE OF DIFFERENT ALGORITHMS, WHEN $N \in [5, 10]$, $D_0^L \in [1, 3]$ DATA UNIT, $J \in [2, 10]$ AND SETTING-1 IS USED ($\alpha_2 = 10$, $d_b = 0.2$)

	SR(%)	DR(%)	SDR(%)	CR(%)
D3QN	94.1	99.8	93.91	3.5
Dueling DQN	93.5	99.7	93.22	4
DDQN	92.5	99.8	92.31	4.3
DQN	90.8	99	89.89	4.2

In addition, we compare the performance of the proposed D3QN algorithm with another benchmark, specifically, the nodes-as-waypoints algorithm, in which the positions of the IoT nodes are the waypoints of the typical UAV's trajectory. The performance comparisons are as follows: 1) if there is no other UAV in the environment, the averaged mission completion time using the benchmark is 112.54 s, while it is 70.95 s with D3QN, indicating that the proposed D3QN algorithm can greatly reduce the time required to change orientation under the kinematic constraints; and 2) in scenarios with 20 other UAVs, CR using the benchmark is 26.4%, while it is 0.6% with D3QN, again indicating that the proposed algorithm can significantly reduce the CR.

V. CONCLUSION

In this work, we have studied the UAV trajectory optimization to maximize the collected data from distributed IoT nodes in a multi-UAV scenario under realistic constraints, e.g., collision-avoidance, mission completion deadline, and kinematic constraints. In establishing the wireless connection, we have taken into account the antenna radiation pattern, path loss, SNR, and largest received-signal-power-based scheduling strategy. We have translated the considered problem into an MDP with parameterized states, permissible actions, and detailed reward functions. D3QN is utilized for learning the policy, without any prior knowledge of the environment (e.g., channel propagation model, locations of the obstacles) and other UAVs (e.g., their missions, movements, and policies). We have shown that the proposed algorithm has high adaptive capability. More specifically, without further training, the offline learned policy can be used for real-time navigation for various missions with different numbers and locations of IoT nodes, different amount of data to be collected, in various scenarios with different number and locations of other UAVs. Through numerical results, we have demonstrated that real-time navigation can be efficiently performed with high SR, high DR and low CR. We also showed that the proposed algorithm can achieve much lower CR in testing compared with the learning algorithm that does not consider collision avoidance. In addition, the proposed algorithm has high tolerance to noisy observations. Furthermore, we have demonstrated that D3QN has better performance than Dueling DQN, DDQN, DQN, and the nodes-as-waypoints algorithm when solving the considered problem.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Y. Zeng, Q. Wu, and R. Zhang, "Accessing from the sky: A tutorial on UAV communications for 5G and beyond," *Proc. IEEE*, vol. 107, no. 12, pp. 2327–2375, Dec. 2019.

[2] Y. Zeng, J. Lyu, and R. Zhang, "Cellular-connected UAV: Potential, challenges, and promising technologies," *IEEE Wireless Commun.*, vol. 26, no. 1, pp. 120–127, Feb. 2019.

[3] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam, and M. Debbah, "A tutorial on UAVs for wireless networks: Applications, challenges, and open problems," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 3, pp. 2334–2360, 3rd Quart., 2019.

[4] C. Liu, M. Ding, C. Ma, Q. Li, Z. Lin, and Y. Liang, "Performance analysis for practical unmanned aerial vehicle networks with LoS/NLoS transmissions," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, May 2018, pp. 1–6.

[5] H. Bayerlein, M. Theile, M. Caccamo, and D. Gesbert, "Multi-UAV path planning for wireless data harvesting with deep reinforcement learning," *IEEE Open J. Commun. Soc.*, vol. 2, pp. 1171–1187, 2021.

[6] M. Mozaffari, W. Saad, M. Bennis, and M. Debbah, "Mobile unmanned aerial vehicles (UAVs) for energy-efficient Internet of Things communications," *IEEE Trans. Wireless Commun.*, vol. 16, no. 11, pp. 7574–7589, Nov. 2017.

[7] U. Challita, W. Saad, and C. Bettstetter, "Interference management for cellular-connected UAVs: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2125–2140, Apr. 2019.

[8] E. Bulut and I. Guevenc, "Trajectory optimization for cellular-connected UAVs with disconnectivity constraint," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, 2018, pp. 1–6.

[9] S. Zhang, Y. Zeng, and R. Zhang, "Cellular-enabled UAV communication: A connectivity-constrained trajectory optimization perspective," *IEEE Trans. Commun.*, vol. 67, no. 3, pp. 2580–2604, Mar. 2019.

[10] E. Vinogradov, H. Sallouha, S. De Bast, M. M. Azari, and S. Pollin, "Tutorial on UAV: A blue sky view on wireless communication," 2019, *arXiv:1901.02306*.

[11] C. Zhan and Y. Zeng, "Aerial–ground cost tradeoff for multi-UAV-enabled data collection in wireless sensor networks," *IEEE Trans. Commun.*, vol. 68, no. 3, pp. 1937–1950, Mar. 2020.

[12] Z. Wang, R. Liu, Q. Liu, J. S. Thompson, and M. Kadoch, "Energy-efficient data collection and device positioning in UAV-assisted IoT," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1122–1139, Feb. 2020.

[13] J. Li *et al.*, "Joint optimization on trajectory, altitude, velocity, and link scheduling for minimum mission time in UAV-aided data collection," *IEEE Internet Things J.*, vol. 7, no. 2, pp. 1464–1475, Feb. 2020.

[14] M. Samir, S. Sharafeddine, C. M. Assi, T. M. Nguyen, and A. Ghrayeb, "UAV trajectory planning for data collection from time-constrained IoT devices," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 34–46, Jan. 2020.

[15] Q. Wu, Y. Zeng, and R. Zhang, "Joint trajectory and communication design for multi-UAV enabled wireless networks," *IEEE Trans. Wireless Commun.*, vol. 17, no. 3, pp. 2109–2121, Mar. 2018.

[16] C. Zhan, Y. Zeng, and R. Zhang, "Energy-efficient data collection in UAV enabled wireless sensor network," *IEEE Wireless Commun. Lett.*, vol. 7, no. 3, pp. 328–331, Jun. 2018.

[17] M. Hua, L. Yang, Q. Wu, and A. L. Swindlehurst, "3D UAV trajectory and communication design for simultaneous uplink and downlink transmission," *IEEE Trans. Commun.*, vol. 68, no. 9, pp. 5908–5923, Sep. 2020.

[18] H. Wang, G. Ren, J. Chen, G. Ding, and Y. Yang, "Unmanned aerial vehicle-aided communications: Joint transmit power and trajectory optimization," *IEEE Wireless Commun. Lett.*, vol. 7, no. 4, pp. 522–525, Aug. 2018.

[19] J. Baek, S. I. Han, and Y. Han, "Energy-efficient UAV routing for wireless sensor networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1741–1750, Feb. 2020.

[20] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proc. AAAI Conf. Artif. Intell.*, vol. 32, 2018, pp. 3207–3214.

[21] M. Yi, X. Wang, J. Liu, Y. Zhang, and B. Bai, "Deep reinforcement learning for fresh data collection in UAV-assisted IoT networks," in *Proc. IEEE INFOCOM Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2020, pp. 716–721.

[22] B. Zhang, C. H. Liu, J. Tang, Z. Xu, J. Ma, and W. Wang, "Learning-based energy-efficient data collection by unmanned vehicles in smart cities," *IEEE Trans. Ind. Informat.*, vol. 14, no. 4, pp. 1666–1676, Apr. 2018.

[23] K. Li, W. Ni, E. Tovar, and A. Jamalipour, "On-board deep Q-network for UAV-assisted online power transfer and data collection," *IEEE Trans. Veh. Technol.*, vol. 68, no. 12, pp. 12215–12226, Dec. 2019.

[24] O. Bouhamed, H. Ghazzai, H. Besbes, and Y. Massoud, "A UAV-assisted data collection for wireless sensor networks: Autonomous navigation and scheduling," *IEEE Access*, vol. 8, pp. 110446–110460, 2020.

[25] S. Fu, Y. Tang, Y. Wu, N. Zhang, H. Gu, C. Chen, and M. Liu, "Energy-efficient UAV enabled data collection via wireless charging: A reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 10209–10219, Jun. 2021.

[26] H. Bayerlein, P. De Kerret, and D. Gesbert, "Trajectory optimization for autonomous flying base station via reinforcement learning," in *Proc. IEEE 19th Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, 2018, pp. 1–5.

[27] Y.-H. Hsu and R.-H. Gau, "Reinforcement learning-based collision avoidance and optimal trajectory planning in UAV communication networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 1, pp. 306–320, Jan. 2022.

[28] X. Liu, Y. Liu, Y. Chen, and L. Hanzo, "Trajectory design and power control for multi-UAV assisted wireless networks: A machine learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 7957–7969, Aug. 2019.

[29] T. Li, W. Liu, Z. Zeng, and N. N. Xiong, "DRLR: A deep reinforcement learning based recruitment scheme for massive data collections in 6G-based IoT networks," *IEEE Internet Things J.*, early access, Mar. 22, 2021, doi: 10.1109/JIOT.2021.3067904.

[30] N. Zhao, Z. Liu, and Y. Cheng, "Multi-agent deep reinforcement learning for trajectory design and power allocation in multi-UAV networks," *IEEE Access*, vol. 8, pp. 139670–139679, 2020.

[31] "Study on enhanced LTE support for aerial vehicles, version 15.0.0," 3GPP, Sophia Antipolis, France, Rep. TR 36.777, Dec. 2017.

[32] J. Chen, D. Raye, W. Khawaja, P. Sinha, and I. Guvenc, "Impact of 3D UWB antenna radiation pattern on air-to-ground drone connectivity," in *Proc. IEEE 88th Veh. Technol. Conf. (VTC-Fall)*, Aug. 2018, pp. 1–5.

[33] Y. F. Chen, M. Liu, M. Everett, and J. P. How, "Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2017, pp. 285–292.

[34] M. Everett, Y. F. Chen, and J. P. How, "Collision avoidance in pedestrian-rich environments with deep reinforcement learning," *IEEE Access*, vol. 9, pp. 10357–10377, 2021.

[35] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.

[36] C. J. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279–292, 1992.

[37] P. Lv, X. Wang, Y. Cheng, and Z. Duan, "Stochastic double deep Q-network," *IEEE Access*, vol. 7, pp. 79446–79454, 2019.

[38] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. 30th AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.

[39] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.

[40] N. Zhao, Y.-C. Liang, D. Niyato, Y. Pei, M. Wu, and Y. Jiang, "Deep reinforcement learning for user association and resource allocation in heterogeneous cellular networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 11, pp. 5141–5152, Nov. 2019.

[41] J. Van Den Berg, S. J. Guy, M. Lin, and D. Manocha, "Reciprocal n-body collision avoidance," in *Robotics Research*. Heidelberg, Germany: Springer, 2011, pp. 3–19.

Xueyuan Wang received the B.S. degree in electrical and electronics engineering from Beijing University of Posts and Telecommunications, Beijing, China, in 2013, and the M.S. degree in electrical engineering and the Ph.D. degree in electrical and computer engineering from Syracuse University, Syracuse, NY, USA, in 2016 and 2021, respectively.

She is currently an Instructor with the School of Computer Science and Artificial Intelligence, Changzhou University, Changzhou, China. Her primary research interests include unmanned aerial vehicles-enabled networks, 5G and beyond communications, Internet of Things networks, and reinforcement learning.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.

M. Cenk Gursoy (Senior Member, IEEE) received the B.S. degree (High Distinction) in electrical and electronics engineering from Bogazici University, Istanbul, Turkey, in 1999, and the Ph.D. degree in electrical engineering from Princeton University, Princeton, NJ, USA, in 2004.

He is currently a Professor with the Department of Electrical Engineering and Computer Science, Syracuse University, Syracuse, NY, USA. His research interests are in the general areas of wireless communications, information theory, communication networks, signal processing, and machine learning.

Dr. Gursoy received an NSF CAREER Award in 2006. More recently, he received the *EURASIP Journal on Wireless Communications and Networking* Best Paper Award, the 2020 IEEE Region 1 Technological Innovation (Academic) Award, The 38th AIAA/IEEE Digital Avionics Systems Conference Best of Session (UTM-4) Award in 2019, the 2017 IEEE PIMRC Best Paper Award, the 2017 IEEE Green Communications & Computing Technical Committee Best Journal Paper Award, the UNL College Distinguished Teaching Award, and the Maude Hammond Fling Faculty Research Fellowship. He was a recipient of the Gordon Wu Graduate Fellowship from Princeton University from 1999 to 2003. He is a member of the editorial boards of IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS and IEEE TRANSACTIONS ON COMMUNICATIONS, and is an Area Editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY. He also served as an Editor of IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING from 2016 to 2021, IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS from 2010 to 2015, IEEE COMMUNICATIONS LETTERS from 2012 to 2014, IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS—Series on Green Communications and Networking (JSAC-SGCN) from 2015 to 2016, *Physical Communication* (Elsevier) from 2010 to 2017, and IEEE TRANSACTIONS ON COMMUNICATIONS from 2013 to 2018. He was the Co-Chair of the 2017 International Conference on Computing, Networking and Communications—Communication QoS and System Modeling Symposium, 2019 IEEE Global Communications Conference (Globecom)—Wireless Communications Symposium, 2019 IEEE Vehicular Technology Conference Fall—Green Communications and Networks Track, and 2021 IEEE Globecom, Signal Processing for Communications Symposium. He is the Aerospace/Communications/Signal Processing Chapter Co-Chair of IEEE Syracuse Section.

Tugba Erpek received the B.S. degree in electrical and electronics engineering from Osmangazi University, Eskisehir, Turkey, in 2005, the M.S. degree in electrical and computer engineering from George Mason University, Fairfax, VA, USA, in 2007, and the Ph.D. degree in electrical and computer engineering from Virginia Tech, Arlington, VA, USA, in 2019.

She is a Lead Scientist of the Networks and Security Division at Intelligent Automation, BlueHalo, Huntsville, AL, USA, where she is also the Network Communications Technical Area Lead. She was an Adjunct Research Professor with the Hume Center, Virginia Tech, Blacksburg, VA, USA. She was developing machine/deep learning algorithms to improve the performance, situational awareness, and security of wireless communications systems. Her research interests are in wireless communications and networks, security, resource allocation, 5G and beyond communications, machine learning, and adversarial machine learning.

Yalin E. Sagduyu (Senior Member, IEEE) received the B.S. degree in electrical and electronics engineering from Bogazici University, Istanbul, Turkey, in 2000, and the M.S. and Ph.D. degrees in electrical and computer engineering from the University of Maryland at College Park, College Park, MD, USA, in 2002 and 2007, respectively.

He is the Director of Networks and Security with Intelligent Automation, BlueHalo, Huntsville, AL, USA. He is a Visiting Research Professor with the Electrical and Computer Engineering Department, University of Maryland. His research interests include wireless communications, networks, security, and machine learning.

Dr. Sagduyu was the recipient of the IEEE HST 2018 Best Paper Award. He chaired workshops at ACM MobiCom, ACM WiSec, IEEE CNS, and IEEE ICNP, was a Track Chair at IEEE PIMRC, IEEE GlobalSIP, and IEEE MILCOM, and was in the organizing committee of IEEE GLOBECOM.

Authorized licensed use limited to: INDIAN INST OF INFO TECH AND MANAGEMENT. Downloaded on February 20,2026 at 08:27:40 UTC from IEEE Xplore. Restrictions apply.